

Makemore

Part 4 - Becoming a Backprop Ninja

Manual backpropagation, one tensor at a time

A comprehensive, exercise-by-exercise study handbook for the fourth video in Andrej Karpathy's *makemore* series — “Building makemore Part 4: Becoming a Backprop Ninja.” This expanded edition explains the **what**, the **why**, and the **how** behind every gradient derivation, every intermediate tensor, and every line of manual backward-pass code in the notebook.

Compiled by: **Shreyas Pradeepkumar Khandale** · GitHub: **sherurox**

Companion to: `build_makemore_backprop_ninja.ipynb`

Edition: Expanded - June 2026

Where this handbook sits in the series

Part 3 took the Part 2 MLP, diagnosed its initialization, fixed it with Kaiming scaling, introduced Batch Normalization, refactored everything into PyTorch-style modules, and then built a six-layer deep network instrumented with diagnostic histograms. Throughout all of that, every gradient was computed for us by a single call to `loss.backward()`. We treated PyTorch's autograd as a black box that magically delivered the right answers.

Part 4 is the lecture where we open the box. The architecture does not change. The training loop does not change. What changes is that we delete the call to `loss.backward()` and write the backward pass ourselves, by hand, on the level of tensors. Karpathy calls this becoming a “backprop ninja,” and the reason for the exercise is that backpropagation is what he calls a *leaky abstraction*: it does not just magically work, and the only way to debug a neural network that is silently broken is to understand what is happening inside the backward pass. By the end of this lecture, every single derivative in the network — from the loss all the way back to the embedding matrix — will have been derived from first principles and implemented in code.

What is actually new compared to Part 3

- **The expanded forward pass.** The two-line forward pass from Part 3 is broken open into roughly 25 intermediate tensors, each one named and isolated. This is what lets us backprop through them one at a time.

- **The cmp utility.** A custom comparison function checks every manually-computed gradient against PyTorch's autograd result for exact and approximate equality, with a printed maximum difference for diagnosis.
- **Exercise 1: backprop through everything manually.** Twenty-five derivations, each a few lines of code, covering the cross-entropy loss, the output linear layer, tanh, BatchNorm, the first linear layer, the embedding lookup, and the view operation.
- **Exercise 2: cross-entropy in one shot.** Pen-and-paper derivation of $d\text{logits} = \text{softmax}(\text{logits}) - \text{one_hot}(Y)$, divided by N . The beautiful three-line replacement for the whole loss subgraph.
- **Exercise 3: BatchNorm in one shot.** The infamous analytical derivation of the BatchNorm backward pass — the hardest single derivation in any lecture in the series, condensed into a single line of code.
- **Exercise 4: training without autograd.** Delete `loss.backward()`, plug in the manual gradients, wrap the whole loop in `torch.no_grad()`, and watch the network train to the same loss as before.
- **The shape-matching shortcut.** A practical trick for working out the backward of matrix multiplications without re-deriving the formula every time — just figure out what shape the result has to be and which transpose makes it work.
- **The push-and-pull intuition.** A geometric way of reading the cross-entropy gradient as forces acting on logits, pulling the correct class up and pushing the incorrect ones down in proportion to the error.

Preface: How to use this handbook

This handbook is the fourth in a coordinated series covering Andrej Karpathy's *makemore* lectures. It is organized as a guided walk through the `build_makemore_backprop_ninja.ipynb` notebook, with each chapter mapping to a coherent group of derivations in the notebook. The format is identical to the Parts 1–3 expanded editions so the four documents can be read side-by-side as one continuous reference.

Reading mode 1: Active study

Part 4 is unique in the series in that it is structured as a sequence of exercises rather than a narrative lecture. The single most effective way to learn this material is to follow Karpathy's explicit recommendation: open the notebook, attempt each gradient derivation yourself before looking at the answer, and only then watch his explanation or read this handbook's walkthrough. This handbook is structured to support that workflow — each chapter begins with the forward-pass line and the shape information you need to attempt the derivation, then presents the answer with a full explanation. If you read the answer first you will lose most of the value of the exercise.

Reading mode 2: Reference lookup

After you have worked through the exercises once, this handbook also functions as a reference. The Table of Contents lists every derivation by name, the Glossary defines every term, and the diagnostic cards in Chapter 28 capture the most common bugs you will encounter when writing manual backward passes in your own projects. Most of the time when an autograd-based network is silently broken, the bug is one of those listed there.

Conventions used in this handbook

The handbook uses a small set of visual conventions consistently throughout the document. They are identical to the conventions in Parts 1–3.

- **Code blocks** appear in a fixed-width font on a warm beige background. Code is reproduced verbatim from the notebook so you can match each block to the cell that produced it.
- **Cell labels** precede every code block in small blue type (for example, “*Manual backward — dlogprobs.*”).
- **Callout boxes** (yellow background, rust-coloured title) highlight Karpathy's spoken framing, important warnings, or moments of insight.
- **Info boxes** (light blue background) contain conceptual definitions and theoretical background. Useful for first-time learners and skimmable for readers already familiar with the material.
- **Part dividers** separate the five major sections with a full-width banner.
- **Diagnostic cards** (Chapter 28) present each common manual-backprop bug in its own clearly bounded card with three labeled rows: symptom, diagnosis, and action.

Prerequisites

This handbook assumes you have completed Parts 1–3 of the makemore series (or are comfortable with the MLP architecture, embeddings, the cross-entropy loss, BatchNorm, and the train/dev/test split). Mathematical prerequisites are modestly heavier than the previous handbooks: you should be comfortable with the chain rule, taking partial derivatives of multivariable functions, and reasoning about how matrix dimensions chain through products. Familiarity with the scalar autograd we built in micrograd (the first video in the series) will help, because every tensor backward in Part 4 is the natural generalization of a scalar backward.

How to extract maximum value from this material

Part 4 is the densest mathematical lecture in the series, and it is also the most rewarding once internalized. The intuitions you build here — about how broadcasts become sums in the backward, how indexing becomes scatter, how matrix multiplications produce a pair of matrix multiplications — are the foundation for understanding every deep learning paper that touches optimization. The single most useful exercise after working through the notebook is to take a small piece of code from a paper or framework you do not understand, identify a forward operation, and derive its backward on paper. Once you can do this confidently, you have arrived.

Contents

Front matter

Preface: How to use this handbook

PART I — Motivation and setup

1. Why we are doing manual backprop in 2026
2. Backprop as a leaky abstraction
3. The starting code — same network, expanded forward pass

PART II — Tools for the journey

4. The `cmp` utility function
5. Non-standard initialization to unmask bugs
6. `retain_grad` and what the forward pass looks like now

PART III — Exercise 1: back through the loss

7. `dlogprobs` — through the mean and indexing
8. `dprobs` — through the element-wise log
9. `dcounts_sum_inv` — broadcasting multiplication
10. `dcounts_sum` and `dcounts` — combining two paths
11. `dnorm_logits` — through `exp`
12. `dlogit_maxes` — the numerical-stability branch
13. `dlogits` — assembling the final logit gradient

PART IV — Exercise 1: back through the network

14. `dh`, `dW2`, `db2` — the output linear layer by shape matching
15. `dhpreact` — through `tanh`
16. `dbngain`, `dbnbias`, `dbnraw` — the BatchNorm scale and shift
17. `dbnvar_inv`, `dbnvar`, `dbndiff2` — the variance subgraph
18. `dbndiff`, `dbnmeani`, `dhprebn` — closing the BatchNorm
19. `dembcat`, `dW1`, `db1` — the first linear layer
20. `demb` and `dC` — the view and the embedding lookup

PART V — Analytical shortcuts and putting it all together

21. Exercise 2: cross-entropy in one shot
22. The push-and-pull intuition for the loss gradient
23. Exercise 3: BatchNorm in one shot
24. The full BatchNorm backward derivation
25. The compact one-line formula

- 26. Exercise 4: training without `loss.backward()`
- 27. Verification and the final loss
- 28. Diagnostic cards: common manual-backprop bugs
- 29. Summary and what Part 5 will introduce
- 30. Glossary

PART I

Motivation and setup

Why we are deleting `loss.backward()` and writing the backward pass by hand in a world where every framework ships with autograd.

1. Why we are doing manual backprop in 2026

Chapter overview

Karpathy opens the lecture with a direct admission: writing the backward pass by hand is no longer how anyone does deep learning in practice. Modern frameworks ship with autograd engines that handle backpropagation correctly and efficiently, and the right thing to do in a production codebase is to call `loss.backward()` and move on. So why are we doing this? This chapter explains the pedagogical motivation: backpropagation is a *leaky abstraction*, and the only way to debug a network whose autograd is silently producing the wrong gradients is to know what the right gradients look like.

We have, at this point in the series, trained a working multilayer perceptron, fixed its initialization, added Batch Normalization, and built a diagnostic toolkit for inspecting activations and gradients. Throughout all of that, every gradient came from a single call:

```
loss.backward()
```

PyTorch's autograd does the rest. It traces the operations we performed on tensors during the forward pass, builds a directed acyclic graph of those operations, and walks the graph in reverse to compute gradients. We have been treating that machinery as a black box. In this lecture, we open the box.

What "becoming a backprop ninja" actually means

The phrase is Karpathy's. The skill it points at is the ability to look at any forward-pass expression involving tensors — matrix multiplication, broadcasting, indexing, normalization, non-linearities — and immediately write down the analytical expression for its gradient with respect to each input. Once you can do this, you can debug any neural network, write custom operations that integrate with autograd, and reason about why training is or is not working. The exercise of doing it manually for our MLP is how you build that skill.

A bit of history

Karpathy includes a short historical interlude that is worth pausing on. About ten years ago, writing the backward pass by hand was not an exotic exercise — it was standard practice. Python was not the dominant language for deep learning; MATLAB was. Autograd engines did not exist in widely-used libraries. Every paper that introduced a new architecture also introduced, by necessity, a hand-derived backward pass for every layer. People used gradient checkers (finite differences) to verify their derivations. The result was a generation of researchers who understood neural networks at a level that is uncommon today — not because they were smarter, but because the tools forced them to.

When autograd became standard with Theano (2012) and TensorFlow (2015) and finally PyTorch (2016), the productivity gain was enormous, but something was lost. It is now possible to build and train networks for years without ever thinking about what backpropagation actually does, and that is exactly the gap Karpathy wants to close in this lecture.

2. Backprop as a leaky abstraction

Chapter overview

An abstraction is *leaky* when it pretends to hide the underlying mechanism but in fact exposes it through bugs, edge cases, and silent failures. Karpathy's claim — which he has written about in a famous blog post titled *Yes, you should understand backprop* — is that backpropagation is one of the most leaky abstractions in modern deep learning. This chapter lays out the cases where the leak shows itself and explains why understanding manual backprop is the only defence.

Where the abstraction leaks

- **Saturated non-linearities** (tanh, sigmoid). When the pre-activation lands in the flat tails, the gradient is multiplied by a number near zero. Autograd reports the right gradient, but the network does not learn. We saw this in Part 3. Unless you know what to look for, you will not see it.
- **Dead ReLU neurons**. If a ReLU neuron's pre-activation is always negative, its gradient is exactly zero, and it never updates. Autograd faithfully computes zero. The network silently has 20% less capacity than it should.
- **Vanishing or exploding gradients in deep networks**. Each layer multiplies the gradient by some factor. If those factors are systematically below or above 1, the gradient at the early layers becomes useless. Autograd reports correct (if tiny or huge) numbers; the network does not train.
- **Loss clipping vs gradient clipping**. Karpathy's blog post highlights a real-world bug from a published codebase: the author tried to make training robust to outliers by clipping the loss at a maximum value. What they actually did was set the *gradient* of outlier examples to zero, which causes those examples to be ignored entirely. Autograd does not flag this; only understanding what clipping does to a gradient catches it.
- **Subtle PyTorch behaviour**. The notebook even contains a small footnote: Karpathy uses `counts_sum ** -1` instead of `1.0 / counts_sum` because he found that PyTorch's backward for division had a tiny numerical discrepancy that prevented bit-exact gradient comparison.

Karpathy's blog post

"Backpropagation does not just make your neural networks just work magically. It is not the case that you can stack up arbitrary Lego blocks of differentiable functions, cross your fingers, and back-propagate and everything is great. Things do not just work automatically. It is a leaky abstraction in the sense that you can shoot yourself in the foot if you do not understand its internals." The full post is titled *Yes, you should understand backprop* and is freely available on Medium.

The defence is knowing what a correct gradient looks like

The pattern across all of the leak cases above is the same: autograd computes *some* gradient, that gradient happens to be technically correct, and the gradient happens to be useless for learning. The only way to spot this is to have a mental model of what gradients *should* look like in a healthy network. That model can only be built by doing manual backprop at least once, slowly, on a simple network, with full

visibility into every intermediate gradient. This lecture is that exercise.

Connecting to Part 3's diagnostic toolkit

Notice the continuity with the previous lecture. In Part 3 we built diagnostic plots for activations, gradients, and update-to-data ratios. Those plots are how you *observe* a leaky-abstraction failure once it has happened. Manual backprop, the subject of this lecture, is how you *understand* why the failure looks the way it does and what to do about it. The two skills are complementary halves of being able to debug a neural network.

3. The starting code - same network, expanded forward pass

Chapter overview

The architecture in this lecture is identical to Part 3's: a two-layer MLP with BatchNorm between the linear and the tanh, trained on the names dataset. What changes is the *level of granularity* at which the forward pass is written. In Part 3 the forward pass was a handful of lines. Here it is expanded into roughly 25 intermediate tensors, each with its own name. This chapter walks through the expanded forward pass and explains why every intermediate is named.

The first several cells of the notebook (imports, data loading, train/dev/test split) are unchanged from Part 3. They are reproduced briefly here and the explanation can be skimmed if you have done the earlier lectures.

Cell - Standard imports and data setup

```
import torch
import torch.nn.functional as F
import matplotlib.pyplot as plt

# (download names.txt, build stoi/itos, build_dataset, train/dev/test - same as Part 3)
vocab_size = 27
block_size = 3
```

The expanded forward pass

Here is where Part 4 diverges. Look at how the forward pass is broken open into named intermediates:

Cell - The forward pass, chunked for manual backprop

```
emb = C[Xb]
embcat = emb.view(emb.shape[0], -1)
# Linear layer 1
hprebn = embcat @ W1 + b1
# BatchNorm layer
bnmean1 = 1/n * hprebn.sum(0, keepdim=True)
bndiff = hprebn - bnmean1
bndiff2 = bndiff ** 2
bnvar = 1/(n-1) * (bndiff2).sum(0, keepdim=True) # Bessel's correction
```

```

bnvar_inv = (bnvar + 1e-5) ** -0.5
bnraw = bndiff * bnvar_inv
hpreact = bngain * bnraw + bnbias
# Non-linearity
h = torch.tanh(hpreact)
# Linear layer 2
logits = h @ W2 + b2
# Cross entropy loss (same as F.cross_entropy(logits, Yb))
logit_maxes = logits.max(1, keepdim=True).values
norm_logits = logits - logit_maxes
counts = norm_logits.exp()
counts_sum = counts.sum(1, keepdims=True)
counts_sum_inv = counts_sum ** -1
probs = counts * counts_sum_inv
logprobs = probs.log()
loss = -logprobs[range(n), Yb].mean()

```

Why every intermediate is named

In Part 3, the forward pass would have read something like `loss = F.cross_entropy(h @ W2 + b2, Yb)`. The single line packs at least eight tensor operations into one expression. For manual backprop, we need to be able to backprop through each of those operations individually, and that means each of them needs its own named output tensor. The naming convention is consistent: every forward tensor gets a name, and its gradient gets the same name prefixed with `d`. So `probs` has gradient `dprobs`, `hpreact` has gradient `dhpreact`, and so on.

Two reasons to use `** -1` instead of `1 /`

The forward pass uses `counts_sum ** -1` rather than the more natural `1.0 / counts_sum`. Karpathy explains in a comment that this is because PyTorch's backward implementation of division produces a tiny numerical error that prevents bit-exact gradient comparison, while the backward of `** -1` is bit-exact. For the purposes of our exercise — checking that manual gradients exactly match PyTorch's — we need bit-exact comparison, so we use the `** -1` form. This is itself an example of a leaky abstraction: the same mathematical operation has different numerical behaviour depending on how you express it.

Cell - The PyTorch backward pass for verification

```

# PyTorch backward pass
for p in parameters:
    p.grad = None
for t in [logprobs, probs, counts, counts_sum, counts_sum_inv,
          norm_logits, logit_maxes, logits, h, hpreact, bnraw,
          bnvar_inv, bnvar, bndiff2, bndiff, hprebn, bnmean1,
          embcat, emb]:
    t.retain_grad()
loss.backward()

```

We run the PyTorch backward pass and store every intermediate's gradient in its `.grad` attribute. That gives us ground truth to check our manual derivations against. The call to `retain_grad()` on every intermediate is necessary because by default PyTorch only keeps gradients for leaf tensors (parameters and inputs); for non-leaf intermediates we have to explicitly ask.

PART II

Tools for the journey

Three pieces of scaffolding that make manual backprop verifiable, debuggable, and possible to perform one operation at a time.

4. The cmp utility function

Chapter overview

Manual backprop is only useful if you can verify that your derivations are correct. The `cmp` utility is a tiny helper function that compares a manually computed gradient against PyTorch's, prints whether they are exactly or approximately equal, and reports the maximum element-wise difference. This chapter introduces the utility and explains why every check it performs matters.

Cell - The `cmp` utility

```
def cmp(s, dt, t):
    ex = torch.all(dt == t.grad).item()
    app = torch.allclose(dt, t.grad)
    maxdiff = (dt - t.grad).abs().max().item()
    print(f'{s:15s} | exact: {str(ex):5s} | approximate: {str(app):5s} | maxdiff: {maxdiff}')
```

What each check is for

- `torch.all(dt == t.grad)` — checks whether every element of our manual gradient is bit-exactly equal to PyTorch's. This is the gold standard. If both expressions are computed the same way numerically, they should produce identical floats.
- `torch.allclose(dt, t.grad)` — checks for approximate equality with PyTorch's default tolerance. If the manual computation uses a different ordering of floating-point operations than PyTorch internally, the results can differ at the last few bits even when mathematically identical. `allclose` tolerates that.
- `(dt - t.grad).abs().max()` — the maximum element-wise absolute difference. When you do not get exact equality, this tells you whether the discrepancy is tiny (e.g. $1e-9$, harmless floating-point noise) or large (e.g. $1e-3$, a real bug).

Why exact vs approximate both matter

Karpathy aims for exact equality wherever possible. Where mathematically the operations are identical but floating-point ordering differs (notably the BatchNorm backward and the cross-entropy one-shot), he settles for approximate equality with a `maxdiff` in the $1e-9$ range. The discipline of first aiming for exact and only accepting approximate when there is a defensible reason catches subtle bugs — for instance, if your sum is missing a dimension or your broadcasting is off by a factor, the difference is usually large enough to show up immediately.

5. Non-standard initialization to unmask bugs

Chapter overview

The notebook initializes several parameters in deliberately non-standard ways. The biases are small random values instead of zero, the BatchNorm parameters have random offsets, and `b1` is included even

though it is spurious in the presence of BatchNorm. None of these choices affect the forward pass meaningfully, but all of them are designed to expose bugs in the backward pass that would otherwise be invisible. This chapter explains the technique.

Cell - Parameter initialization (note the * 0.1 multipliers)

```
n_embd = 10
n_hidden = 64
g = torch.Generator().manual_seed(2147483647)

C = torch.randn((vocab_size, n_embd), generator=g)
# Layer 1
W1 = torch.randn((n_embd * block_size, n_hidden), generator=g) * (5/3) / ((n_embd * block_size) ** 0.5)
b1 = torch.randn(n_hidden, generator=g) * 0.1 # spurious but kept on purpose
# Layer 2
W2 = torch.randn((n_hidden, vocab_size), generator=g) * 0.1
b2 = torch.randn(vocab_size, generator=g) * 0.1
# BatchNorm parameters
bngain = torch.randn((1, n_hidden)) * 0.1 + 1.0
bnbias = torch.randn((1, n_hidden)) * 0.1
```

Why random small initialization, not zero

If `b1` were initialized to exactly zero, many of the gradient expressions involving it would simplify in ways that mask bugs. For instance, a multiplication by zero in the wrong place would silently produce a correct-looking result. By initializing to small random values instead — values close enough to zero that the forward pass behaves sensibly, but nonzero enough that no gradient term vanishes spuriously — we force every backward derivation to actually do its work. If our manual gradient is wrong, the difference shows up in the `maxdiff` output of `cmp`.

Why keep `b1` at all

We saw in Part 3 that the bias of a Linear layer immediately preceding a BatchNorm is mathematically useless — BatchNorm's mean subtraction cancels it out. So why include `b1` here? Karpathy says “just for fun.” The serious reason is that we want to manually compute and verify `db1`, and the only way to do that is to have `b1` in the parameter list. The gradient turns out to be non-zero in our manual computation and matches PyTorch's, even though both gradients are technically “wasted” from a learning perspective.

6. `retain_grad` and what the forward pass looks like now

Chapter overview

PyTorch's autograd, by default, only retains gradients on leaf tensors — the parameters and the inputs. Gradients on intermediate tensors (everything in the middle of the forward pass) are computed during `.backward()` but discarded immediately after they are used to propagate further back. For manual backprop verification, we need those intermediate gradients to compare against, so we have to explicitly

request that they be kept. This chapter explains the `retain_grad()` mechanism.

Before calling `loss.backward()`, we iterate over every intermediate tensor in the forward pass and mark it for gradient retention:

Cell - Retain gradients on every intermediate

```
for t in [logprobs, probs, counts, counts_sum, counts_sum_inv,
         norm_logits, logit_maxes, logits, h, hpreact, bnraw,
         bnvar_inv, bnvar, bndiff2, bndiff, hprebn, bnmean1,
         embcat, emb]:
    t.retain_grad()
```

Leaf tensors vs non-leaf tensors

A *leaf tensor* in PyTorch is one that was created directly (by `torch.randn`, for instance) and not as the output of an operation on other tensors. Parameters and inputs are leaves. *Non-leaf tensors* are produced by operations — everything from `x @ W` to `x + 1` creates a non-leaf. By default, only leaves keep their gradients after backward, because that is all autograd needs to update parameters. Manual backprop is a special case that needs the intermediates too, and `retain_grad()` is how you opt in per-tensor.

The plan from here

We now have everything we need: an expanded forward pass with named intermediates, a `cmp` utility for verification, parameter initialization that will unmask any bugs, and PyTorch's gradients computed on every intermediate as ground truth. Part III of this handbook walks through the manual derivation of every gradient in the loss subgraph (logprobs down through logits). Part IV continues through the network body (h down through C). Part V then revisits two of the derivations — cross-entropy and BatchNorm — and replaces them with elegant analytical one-shot formulas.

PART III

Exercise 1: back through the loss

Manual gradient derivations through the cross-entropy subgraph — seven chapters, seven gradients, every one verified against PyTorch.

7. dlogprobs - through the mean and indexing

Chapter overview

The first manual derivation is the gradient of the loss with respect to `logprobs`. The forward operation we are inverting is the line `loss = -logprobs[range(n), Yb].mean()`, which combines two operations: an indexing that plucks out the log-probability assigned to the correct character of each example, and a mean that averages those plucked-out values into a single scalar loss. This chapter walks through both operations and produces the gradient.

Reasoning through the math

The loss is computed in two stages. First, indexing: from the $(n, 27)$ tensor `logprobs`, we extract one element per example — specifically, `logprobs[i, Yb[i]]` for each i . This gives us a vector of length n . Second, the mean: $(-1) * \text{mean}(\text{plucked})$. The mean of n values is the sum divided by n , so the loss can be written as $-(a + b + c + \dots) / n$.

Differentiating a mean

If $\text{loss} = -(a + b + c) / 3$, then $d\text{loss}/da = -1/3$. By the same logic, the derivative of the loss with respect to each of the n plucked-out values is exactly $-1/n$. This is the simplest case of the rule: the gradient of a mean over n elements distributes evenly, with each element receiving $1/n$ times the gradient of the mean.

Reasoning through the indexing

Only n of the $32 \times 27 = 864$ elements of `logprobs` were plucked out by the indexing. The other $864 - n$ elements never reached the loss. Their gradient must therefore be exactly zero: if you change them, the loss does not change, so $d\text{loss} / d(\text{those elements}) = 0$. The gradient tensor `dlogprobs` is shape $(n, 27)$ with $-1/n$ at exactly the n positions $(i, Yb[i])$ and zeros everywhere else.

Manual backward - dlogprobs

```
dlogprobs = torch.zeros_like(logprobs)
dlogprobs[range(n), Yb] = -1.0 / n
cmp('logprobs', dlogprobs, logprobs)
```

Three lines. The first creates a zero tensor in the shape of `logprobs`. The second uses the exact same indexing pattern as the forward pass to write $-1/n$ into the right cells. The third compares to PyTorch's gradient. The output is exact: True | approximate: True | maxdiff: 0.0 — bit-exact match.

The pattern that will recur

Indexing in the forward pass becomes *scatter* in the backward pass. Wherever an element was plucked out, that exact position in the gradient tensor receives the upstream gradient. This same pattern reappears in Chapters 12 (the max indexing for numerical stability) and 20 (the embedding lookup).

8. dprobs - through the element-wise log

Chapter overview

Next we backprop one step further: from `logprobs` back to `probs`. The forward operation is `logprobs = probs.log()`, an element-wise natural logarithm. The derivative of $\log(x)$ with respect to x is $1/x$, so the chain rule gives us a one-liner.

By the chain rule: `dprobs = (1 / probs) * dlogprobs`. The $1/\text{probs}$ is the local derivative of the log operation, applied element-wise, and the multiplication by `dlogprobs` is the chain rule passing the upstream gradient through.

Manual backward - dprobs

```
dprobs = (1.0 / probs) * dlogprobs
cmp('probs', dprobs, probs)
```

Reading the result intuitively

Notice what this gradient does in practice. For examples where `probs` is close to 1 — the network is correctly very confident in the right answer — we get $1/1 = 1$, so `dprobs` just passes `dlogprobs` through. For examples where `probs` is tiny — the network is confidently wrong — we get $1/\text{tiny} = \text{huge}$, and the gradient is boosted enormously. The network learns most aggressively from the examples it is most wrong about, which is exactly what we want.

The pattern of element-wise backward passes

Any element-wise forward operation produces an element-wise backward operation. Write down the scalar derivative of the operation (here, $d \log(x) / dx = 1/x$), apply it element-wise to the forward input, and multiply by the upstream gradient. This same pattern appears for `tanh` (Chapter 15) and `exp` (Chapter 11).

9. dcounts_sum_inv - broadcasting multiplication

Chapter overview

Now the derivations get more interesting. The forward line is `probs = counts * counts_sum_inv`, but the shapes do not match: `counts` is (32, 27) and `counts_sum_inv` is (32, 1). PyTorch broadcasts the second tensor across the column dimension to make the multiplication work. This chapter explains how broadcasting affects the backward pass and how to compute the gradient of the broadcast tensor.

The broadcasting forward, expanded

Even though we wrote `probs = counts * counts_sum_inv`, what PyTorch actually does is replicate `counts_sum_inv` 27 times across the column dimension and then do an element-wise multiply. So `counts_sum_inv[i, 0]` effectively gets multiplied by 27 different elements of `counts` on row `i`, producing 27 different elements of `probs` on that row.

The rule: broadcast in forward = sum in backward

Whenever a tensor is broadcast (replicated) along some axis in the forward pass, its gradient in the backward pass is the *sum* over that same axis. Intuition: the original element contributed to many output elements, so each of those output gradients flows back to that single input. The chain rule says we add up all the contributions. This is one of the most useful single rules in manual backprop, and you will see it applied repeatedly in the chapters that follow.

Applying this rule: `dcounts_sum_inv` is the sum across the column dimension of `counts * dprobs` (the gradient through the multiplication, where `counts` is the local derivative since $d(a*b)/db = a$).

Manual backward - `dcounts_sum_inv`

```
dcounts_sum_inv = (counts * dprobs).sum(1, keepdim=True)
cmp('counts_sum_inv', dcounts_sum_inv, counts_sum_inv)
```

`keepdim=True` is important: it preserves the `(32, 1)` shape rather than collapsing to `(32,)`. Without it, the next gradient computation that uses `dcounts_sum_inv` would fail or broadcast incorrectly.

10. `dcounts_sum` and `dcounts` - combining two paths

Chapter overview

Two derivations in one chapter, because they share a structural feature: `counts` feeds into two downstream operations (it produces `counts_sum` via summing, and it directly produces `probs` via the multiplication). When a tensor is used in multiple places in the forward pass, its gradient is the *sum* of the gradients flowing back from each use. This chapter handles the first of many such multi-path tensors.

`dcounts_sum`: through the negative power

The forward operation is `counts_sum_inv = counts_sum ** -1`. The derivative of x^{-1} is $-x^{-2}$, so by the chain rule:

Manual backward - `dcounts_sum`

```
dcounts_sum = (-counts_sum ** -2) * dcounts_sum_inv
cmp('counts_sum', dcounts_sum, counts_sum)
```

`dcounts`: combining contributions from two paths

`counts` is used twice in the forward pass. First, it is multiplied by `counts_sum_inv` to produce `probs`. Second, it is summed across the column dimension to produce `counts_sum`. Each of these is a separate gradient contribution, and the total `dcounts` is their sum.

The first contribution comes through the multiplication:

```
dcounts = counts_sum_inv * dprobs    # broadcasts (32,1) over (32,27)
```

The second contribution comes through the sum operation:

```
dcounts += torch.ones_like(counts) * dcounts_sum
```

Why ones_like for the sum

When the forward operation is `counts_sum = counts.sum(1)`, every element of `counts` contributes equally to `counts_sum` — the local derivative is 1. So the gradient through a sum is the upstream gradient replicated (broadcast) back into the shape of the input. `torch.ones_like(counts) * dcounts_sum` performs that replication. This is the dual of the earlier rule: *sum in forward = broadcast in backward*.

Combined - dcounts (both contributions)

```
dcounts = counts_sum_inv * dprobs
dcounts += torch.ones_like(counts) * dcounts_sum
cmp('counts', dcounts, counts)
```

11. dnorm_logits - through exp

Chapter overview

Backing up one more step, we have `counts = norm_logits.exp()`, an element-wise exponential. The derivative of $\exp(x)$ is $\exp(x)$ itself — the only function whose derivative equals itself — which makes this derivation the most compact in the entire lecture.

By the chain rule: `dnorm_logits = exp(norm_logits) * dcounts = counts * dcounts`. We reuse the already-computed `counts` tensor rather than recomputing the exponential.

Manual backward - dnorm_logits

```
dnorm_logits = counts * dcounts
cmp('norm_logits', dnorm_logits, norm_logits)
```

Why exp has such a clean backward

$d/dx \exp(x) = \exp(x)$ is the defining property of the exponential function. In manual backprop, this means whenever you take the `exp` of a tensor in forward, you can reuse the forward output tensor as the local derivative in backward — no recomputation needed. Modern frameworks exploit this even when you do not write the backward explicitly: operations that have access to their forward output (rather than recomputing it) are cheaper to backprop through.

12. dlogit_maxes - the numerical-stability branch

Chapter overview

The forward pass subtracts the per-row maximum from the logits before exponentiating: `norm_logits = logits - logit_maxes`. This is purely for numerical stability — mathematically the softmax is invariant to a shift — but logically there are two paths from `logits` back to the loss: a direct one and one through `logit_maxes`. This chapter computes the gradient on the `logit_maxes` branch. It should be zero in

theory; in practice it is extremely small but nonzero due to floating-point rounding.

Two parts to this gradient

First, the gradient of `norm_logits = logits - logit_maxes` with respect to `logit_maxes`: the local derivative is `-1`, so we negate `dnorm_logits`. Second, broadcasting: `logit_maxes` has shape `(32, 1)` but was broadcast over 27 columns when subtracted, so we have to sum across columns when going backward.

Manual backward - `dlogit_maxes`

```
dlogit_maxes = (-dnorm_logits).sum(1, keepdim=True)
cmp('logit_maxes', dlogit_maxes, logit_maxes)
```

Why this gradient should be (but isn't quite) zero

Mathematically, shifting every logit by the same constant does not change softmax, so changing `logit_maxes` by any amount should not change the loss. The gradient should therefore be exactly zero. In practice, we see values like `1e-9` or `1e-10`. This is purely floating-point error: the forward computation routes through different operations depending on whether the subtraction was performed or not, and the order of those operations introduces tiny rounding differences. `cmp` still reports `exact: True` because PyTorch's autograd produces the same tiny errors as our manual computation.

13. dlogits - assembling the final logit gradient

Chapter overview

We now combine the two paths from `logits`: the direct path through subtraction and the indirect path through the maximum operation. The result is the gradient `dlogits`, which is the entry point for the gradients flowing into the rest of the network. This chapter handles the second path (through the `max`) and combines it with the first.

The max operation has a scatter-like backward

The forward operation `logit_maxes = logits.max(1, keepdim=True).values` picks the largest element from each row. Going backward, the gradient flowing into `logit_maxes` needs to be scattered back to the exact positions where the maximum values originated. Every other position gets zero gradient, because changing those values (as long as they stay below the max) does not change the output.

Manual backward - `dlogits` (combining both paths)

```
# Path 1: direct contribution through the subtraction
dlogits = dnorm_logits.clone()

# Path 2: scatter contribution through the max
dlogits += F.one_hot(logits.max(1).indices, num_classes=logits.shape[1]) * dlogit_maxes

cmp('logits', dlogits, logits)
```

Reading the two lines:

- `dlogits = dnorm_logits.clone()` — the direct path. The local derivative of logits - logit_maxes with respect to logits is 1, so the gradient passes through. We clone to avoid aliasing.
- `F.one_hot(logits.max(1).indices, num_classes=...) * dlogit_maxes` — the scatter through the max. `F.one_hot` creates a (32, 27) matrix with a 1 at the position of each row's maximum, and we broadcast-multiply by the (32, 1) gradient on the maxes. The result has the gradient deposited at exactly the right positions.

Why this is the entry point to the rest of the network

Every gradient we have computed so far — `dlogprobs`, `dprobs`, `dcounts_sum_inv`, `dcounts_sum`, `dcounts`, `dnorm_logits`, `dlogit_maxes` — lives inside the cross-entropy subgraph. `dlogits` is the first gradient that exits the loss and enters the rest of the network. From here onward we propagate through the output linear layer, `tanh`, `BatchNorm`, the first linear layer, and finally to the embedding matrix and inputs. Part IV picks up there.

PART IV

Exercise 1: back through the network

From dlogits all the way to dC: matrix calculus by shape, tanh, BatchNorm's intricate web, and the indexing operation that produced the embeddings.

14. dh, dW2, db2 - the output linear layer by shape matching

Chapter overview

Backpropagating through a matrix multiplication is the single most important manual-backprop skill, because it shows up in every neural network. Karpathy explains it twice: once through first-principles derivation on a small two-by-two example, and once through a practical shape-matching trick that lets you write down the answer without re-deriving from scratch. This chapter covers both, and applies the result to the output linear layer $\text{logits} = h @ W2 + b2$.

The first-principles derivation

Take a small two-by-two example: $D = A @ B + C$, where A, B, and D are 2x2 matrices and C is a length-2 bias broadcast across rows. Writing out the entries of D in terms of A, B, C and differentiating each $D[i, j]$ with respect to each $A[k, l]$ gives expressions like $dD[i, j]/dA[i, j] = B[j, j]$. Collecting these by the chain rule produces:

```
dA = dD @ B.T
dB = A.T @ dD
dC = dD.sum(0)
```

These are the three formulas. dA is the upstream gradient matrix-multiplied by the transpose of B. dB is the transpose of A matrix-multiplied by the upstream gradient. dC is the sum across the broadcast dimension. These hold for matrices of any size, not just 2x2.

The shape-matching shortcut

Karpathy admits he cannot remember the formulas. Instead, he uses a trick that produces the correct backward without re-deriving anything. The trick has three steps:

- **1. Know the shape of the result.** dA must have the same shape as A. Same for dB and dC. This is a hard constraint.
- **2. Know that the backward involves a matrix multiplication.** Because matrix multiplication is the forward, transposes-of-products is the backward.
- **3. Find the unique combination of inputs and transposes that produces the target shape.** There is usually only one combination of dD, A, B, and their transposes that yields the right output shape.

Worked example: deriving dh by shape matching

We want dh. The shape of h is (32, 64). What can we multiply to get (32, 64)? We have dlogits shape (32, 27) and W2 shape (64, 27). To produce (32, 64), we need $(32, 27) @ (27, 64)$, which means $d\text{logits} @ W2.T$. That is the answer. No formula memorization needed.

Applying the shortcut to our three quantities:

Manual backward - dh, dW2, db2

```
dh = dlogits @ W2.T
dW2 = h.T @ dlogits
db2 = dlogits.sum(0)
```

```
cmp('h', dh, h)
cmp('W2', dW2, W2)
cmp('b2', db2, b2)
```

All three pass with exact equality. The pattern of matrix-multiplication backward will reappear in Chapter 19 when we handle the first linear layer.

15. dhpreact - through tanh

Chapter overview

`h = torch.tanh(hpreact)` is the next operation backward. We saw the tanh derivative in micrograd: $d \tanh(x) / dx = 1 - \tanh(x)^2$. Because we already have the forward output `h`, we can compute the local derivative as $1 - h^2$ directly without recomputing the tanh.

Manual backward - dhpreact

```
dhpreact = (1.0 - h ** 2) * dh
cmp('hpreact', dhpreact, hpreact)
```

Why the derivative is written in terms of the output

Many references write the tanh derivative as $\text{sech}^2(x)$, where x is the input. We instead write it as $1 - \tanh(x)^2$, which is in terms of the output. Both expressions are mathematically equivalent, but the output-based form is much more useful in practice: during the backward pass, we already have the tanh output in memory (we computed it during forward), so we can reuse it. The input form would require us to recompute or keep around `hpreact` separately. This is a recurring pattern in efficient backward passes — reuse forward outputs whenever possible.

16. dbngain, dbnbias, dbnraw - the BatchNorm scale and shift

Chapter overview

We now enter the BatchNorm subgraph. The first line we encounter going backward is `hpreact = bngain * bnraw + bnbias`. This is a straightforward scale and shift, similar to a linear layer, with one wrinkle: `bngain` and `bnbias` have shape $(1, n_hidden)$ while `bnraw` has shape $(32, n_hidden)$, so the gain and bias are broadcast across the batch dimension. This means their backward involves a sum over the batch dimension.

Manual backward - dbngain, dbnbias, dbnraw

```
dbngain = (bnraw * dhpreact).sum(0, keepdim=True)
```

```

dbnbias = dhpreact.sum(0, keepdim=True)
dbnraw = bngain * dhpreact

cmp('bngain', dbngain, bngain)
cmp('bnbias', dbnbias, bnbias)
cmp('bnraw', dbnraw, bnraw)

```

Reading the three lines

- `dbngain = (bnraw * dhpreact).sum(0, keepdim=True)` — The local derivative of `bngain * bnraw + bnbias` with respect to `bngain` is `bnraw`. Multiply by upstream and sum over the broadcast dimension (axis 0, the batch).
- `dbnbias = dhpreact.sum(0, keepdim=True)` — The local derivative with respect to `bnbias` is 1 (it is an additive offset). Multiplied by upstream gives `dhpreact`, and summed over the broadcast dimension.
- `dbnraw = bngain * dhpreact` — The local derivative with respect to `bnraw` is `bngain`. The broadcast goes the other way here (gain is broadcast over batch when multiplying), so the multiplication is element-wise across the batch dimension.

Notice the keepdim asymmetry

`dbngain` and `dbnbias` use `keepdim=True` on their sum, while `dbnraw` does not need any `keepdim` because no summation is happening. This pattern reappears throughout BatchNorm's backward. The rule is simple: if you are computing the gradient of a tensor whose forward shape had a singleton (broadcast) dimension, keep that dimension in the gradient. If you do not, the next computation might broadcast incorrectly.

17. dbnvar_inv, dbnvar, dbndiff2 - the variance subgraph

Chapter overview

We now backprop through the chain that computes the variance: starting from `bnraw = bndiff * bnvar_inv`, we work back through the inverse square root, the variance, and the squared differences. Each step uses one of the patterns we have already seen, but combined the chain becomes intricate. This chapter handles three of those intermediates in order.

dbnvar_inv: broadcast multiplication

`bnraw = bndiff * bnvar_inv`, with shapes (32, n) and (1, n). The gradient on `bnvar_inv` requires summing across the batch dimension.

```
dbnvar_inv = (bndiff * dbnraw).sum(0, keepdim=True)
```

dbnvar: through the inverse square root

`bnvar_inv = (bnvar + 1e-5) ** -0.5`. The derivative is `-0.5 * (bnvar + 1e-5) ** -1.5`, which is element-wise.

`dbnvar = (-0.5 * (bnvar + 1e-5) ** -1.5) * dbnvar_inv`

dbndiff2: through the sum

`bnvar = (1/(n-1)) * bndiff2.sum(0, keepdim=True)`. We multiply by the constant `1/(n-1)` (Bessel's correction) and broadcast back across the batch dimension.

`dbndiff2 = (1.0 / (n - 1)) * torch.ones_like(bndiff2) * dbnvar`

Bessel's correction explained

`bnvar = (bndiff ** 2).sum(0) / (n - 1)` rather than `/ n`. The `n - 1` denominator is called Bessel's correction, and it provides an unbiased estimate of the population variance from a finite sample. PyTorch's `torch.var` defaults to Bessel's correction (via the `unbiased=True` default), which is why this notebook also uses `n - 1`. The choice does not significantly affect the network's behaviour but does affect the exact numerical values of gradients, so we have to match PyTorch's convention if we want bit-exact comparison.

18. dbndiff, dbnmean1, dhprebn - closing the BatchNorm

Chapter overview

We have arrived at `bndiff`, which is used in two places: it produces `bnraw` via multiplication with `bnvar_inv`, and it produces `bndiff2` via the square operation. Like `counts` in Chapter 10, this requires combining gradient contributions from two paths. After that we walk the gradient back to `bnmean1` and finally to `hprebn`, which itself receives contributions from two paths.

dbndiff: two contributions

First contribution: through the multiplication that produces `bnraw`.

`dbndiff = bnvar_inv * dbnraw`

Second contribution: through the square operation that produces `bndiff2`. The derivative of `x ** 2` is `2x`, applied element-wise.

`dbndiff += (2.0 * bndiff) * dbndiff2`

dbnmean1: the mean subtraction

`bndiff = hprebn - bnmean1`. The local derivative with respect to `bnmean1` is `-1`, and `bnmean1` is broadcast over the batch, so we sum across that dimension.

`dbnmean1 = (-dbndiff).sum(0, keepdim=True)`

dhprebn: two contributions, again

hprebn is used twice: in the subtraction that produces bndiff, and in the mean that produces bnmean_i. Two gradient contributions, summed.

Manual backward - dhprebn

```
# Contribution 1: direct, through bndiff = hprebn - bnmean_i
dhprebn = dbndiff.clone()

# Contribution 2: through bnmean_i = (1/n) * hprebn.sum(0)
dhprebn += (1.0 / n) * torch.ones_like(hprebn) * dbnmean_i

cmp('hprebn', dhprebn, hprebn)
```

Why BatchNorm backward gets so hairy

We just chained through eight intermediate gradients to backprop through one BatchNorm layer (dbnraw, dbnvar_inv, dbnvar, dbndiff2, dbndiff, dbnmean_i, dhprebn). Every intermediate brought its own multiplication, sum, or broadcast pattern. This is precisely why Exercise 3 (Chapter 23) replaces all of these with a single line of analytical math — the chain is correct, but it is wasteful and error-prone. The exercise is worth doing once to appreciate the complexity, and once is enough.

19. dembcat, dW1, db1 - the first linear layer

Chapter overview

Backing up further, the line we encounter is $\text{hprebn} = \text{embcat} @ \text{W1} + \text{b1}$. This is the same pattern as the output linear layer in Chapter 14 — matrix multiplication plus a broadcast bias — so we can apply the same shape-matching trick to derive the backward without re-deriving the formula.

Manual backward - dembcat, dW1, db1

```
dembcat = dhprebn @ W1.T
dW1      = embcat.T @ dhprebn
db1      = dhprebn.sum(0)

cmp('embcat', dembcat, embcat)
cmp('W1',     dW1,     W1)
cmp('b1',     db1,     b1)
```

Identical structure to Chapter 14, with different shapes. dembcat is (32, 30), dW1 is (30, n_hidden), db1 is (n_hidden,). The shape-matching trick produces the right answer in seconds without any matrix-calculus derivation. Karpathy emphasizes this point: once you have done one linear-layer backward, you have done all of them. The shape constraints uniquely determine the formula.

Why this skill scales to any architecture

Every modern neural network is built out of a small number of primitive operations: matrix multiplication, element-wise operations, broadcasting, reductions (sum, mean, max), and indexing. Once you can backprop through each of these primitives, you can backprop through any combination of them. A Transformer is just a particular combination of these primitives stacked many times. An LSTM is another combination. CNN, GNN, ViT — all the same primitives. The fluency you are building right now applies to everything.

20. demb and dC - the view and the embedding lookup

Chapter overview

The final two derivations bring the gradient back to the embedding matrix C. First we undo the `.view()` reshape that flattened the embeddings into a single vector per example. Then we undo the indexing that selected rows of C in the first place. The reshape is trivial; the indexing is the trickiest backward in the entire lecture because it involves scattering gradients into positions that may be repeated.

demb: undoing `.view()`

`embcat = emb.view(emb.shape[0], -1)` reshapes `emb` from (32, 3, 10) to (32, 30) without copying data. The backward operation is simply to reshape `dembcat` back to `emb`'s original shape.

```
demb = dembcat.view(emb.shape)
cmp('emb', demb, emb)
```

dC: scatter-add through the embedding lookup

Now the hard one. `emb = C[Xb]` indexes into the (27, 10) embedding matrix C using the (32, 3) integer tensor Xb. Every element of Xb is a character index, and the corresponding row of C is plucked out. The resulting `emb` has shape (32, 3, 10) — for each of 32 examples, three context positions, ten embedding dimensions.

Going backward: for each (example, position) pair in `emb`, we have a 10-dimensional gradient that needs to be added to the corresponding row of `dC`. The catch is that the *same* row of C may have been used many times across the batch — the letter 'a' shows up many times. Every use produces its own gradient contribution, and all of them must be summed into the single row of `dC`.

The dual of indexing is scatter-add

When the forward operation gathers elements (indexing produces a copy of selected entries), the backward operation must scatter gradients back to where they came from. When the same source element is gathered multiple times, the corresponding gradients must accumulate — this is *scatter-add*, not scatter-replace. PyTorch has built-in functions for this (`torch.index_add_` and similar), but Karpathy implements it with an explicit loop for clarity.

Manual backward - dC (explicit loop)

```
dC = torch.zeros_like(C)
for k in range(Xb.shape[0]):
    for j in range(Xb.shape[1]):
        ix = Xb[k, j]
        dC[ix] += demb[k, j]

cmp('C', dC, C)
```

The outer loop iterates over examples, the inner loop over context positions. For each (k, j) pair, we find which row of C was used (`ix = Xb[k, j]`) and add the 10-dimensional gradient `demb[k, j]` to that row. The `+=` is essential: without it, repeated uses of the same row would overwrite each other and we would lose all but the last gradient contribution.

Vectorized alternatives exist but are harder to read

PyTorch's `scatter_add_` and `index_add_` perform exactly this operation without an explicit Python loop, and they are faster on GPU. Karpathy uses the loop for pedagogical clarity. For production code, prefer the vectorized form. Either way, this is the operation that completes Exercise 1: we have now backpropagated through every line of the forward pass and every gradient matches PyTorch's bit-exactly (or within floating-point noise where bit-exactness is not possible).

PART V

Analytical shortcuts and conclusion

Cross-entropy and BatchNorm in one shot each, training without autograd, and a glossary that closes out the series so far.

21. Exercise 2: cross-entropy in one shot

Chapter overview

Exercise 1 produced the right gradients, but it required seven separate derivations through the cross-entropy loss subgraph (Chapters 7–13). That is wasteful. If we look at the mathematical expression for the cross-entropy loss as a function of the logits and apply calculus on paper, the gradient simplifies dramatically. This chapter walks through the analytical derivation and reveals one of the most beautiful results in deep learning: $d\text{logits} = (\text{softmax}(\text{logits}) - \text{one_hot}(Y)) / N$.

The mathematical expression

For a single example with true class y and logits l , the cross-entropy loss is:

$L = -\log(p_y)$
where $p_i = \exp(l_i) / \sum_j \exp(l_j)$ (the softmax)

We want dL / dl_i for every i . Taking the derivative of the logarithm of the softmax, splitting into the case $i = y$ versus $i \neq y$, and simplifying, terms cancel beautifully and the result is:

$dL/dl_i = p_i - 1\{i = y\}$

where $1\{i = y\}$ is 1 when i equals the true class, 0 otherwise

In words: the gradient is the softmax minus a one-hot

For every example, the gradient on the logits is the softmax probability vector minus a one-hot encoding of the correct answer. If the network is perfectly confident in the correct answer (softmax already 1 at the correct position, 0 elsewhere), the gradient is exactly zero. If the network is uniform (softmax = $1/K$ everywhere), the gradient is $1/K$ for incorrect classes and $1/K - 1$ for the correct class. The magnitudes are bounded between 0 and 1, regardless of how wrong the network is.

Implementing it in three lines

Karpathy's solution, after dividing by the batch size N to match the mean reduction:

Manual backward - dlogits in one shot

```
dlogits = F.softmax(logits, 1)
dlogits[range(n), Yb] -= 1
dlogits /= n
```

```
cmp('logits', dlogits, logits)
```

Three lines, replacing seven derivations. Note that the verification reports approximate: True with a maxdiff of about $6e-9$ — not bit-exact, because the analytical computation routes through different floating-point operations than the chain through Exercise 1 did. The mathematical answer is identical; only the floating-point ordering differs.

Why production frameworks fuse this

Every modern deep learning framework implements cross-entropy as a single fused operation, with this analytical backward, rather than building it up from softmax, log, and NLL primitives. The reasons are exactly what we just showed: fewer operations (faster), more numerical stability (no intermediate softmax values to overflow), and a backward pass that is one tensor operation instead of seven. This is what `F.cross_entropy` does internally.

22. The push-and-pull intuition for the loss gradient

Chapter overview

Once you have the formula $\text{dlogits} = (\text{softmax}(\text{logits}) - \text{one_hot}(Y)) / N$, there is a wonderful geometric way to read it. The gradient is a vector of forces acting on each logit, pulling the correct class upward and pushing every incorrect class downward, with the magnitude of each force proportional to how wrong the network was. This chapter develops the intuition and connects it to what training is actually doing.

Reading the gradient as forces

For each example, `dlogits` is a 27-element vector that sums to zero (because the softmax sums to 1 and the one-hot also sums to 1). The positive entries say “this logit should decrease” (gradient descent moves opposite to the gradient), and the single negative entry — at the correct class position — says “this logit should increase.” Because the entries sum to zero, the total push downward exactly equals the total pull upward. The total “tension” is conserved.

The gradient as a pulley system

Karpathy uses this analogy in the lecture. Imagine the logits hanging from the top of the network, and gradient descent as a system of pulleys that pulls them in proportion to the error. When the network is correct, no pulling is needed and the system is at rest. When the network is very wrong — say, it assigns 90% probability to the wrong character — the wrong logit is pulled down hard and the correct one is pulled up by the same amount. The pulley system propagates that tension back through the weights, which slowly give way and the network learns.

Why this matters for understanding training

Notice the self-correcting property: examples the network gets right contribute no gradient (the softmax already matches the one-hot, their difference is zero). Examples the network gets wrong contribute gradients proportional to *how* wrong. This means later in training, when most examples are correct, the gradient signal is dominated by the few remaining hard examples — exactly the right behaviour. Early in training, when most examples are wrong, the network gets a strong, consistent signal across the entire batch. The cross-entropy loss is doing this work for free, with no per-example weighting needed.

23. Exercise 3: BatchNorm in one shot

Chapter overview

Exercise 1 also required eight separate derivations through BatchNorm (Chapters 16–18). Just as with cross-entropy, the analytical derivation produces a much simpler closed-form expression. The math is significantly more involved than for cross-entropy — Karpathy describes filling two sheets of paper — but the resulting one-line formula is compact and elegant. This chapter sets up the problem; Chapter 24 walks through the derivation and Chapter 25 presents the final formula.

The forward operation, written compactly

Treating BatchNorm as a single mathematical expression rather than a chain of operations:

```
hpreact = bngain * (hprebn - hprebn.mean(0, keepdim=True))
           / torch.sqrt(hprebn.var(0, keepdim=True, unbiased=True) + 1e-5)
           + bnbias
```

We want $dhprebn$ given $dhpreact$. We will ignore the gradients on $bngain$ and $bnbias$ for this exercise (those took simple form in Chapter 16 and we keep them as they were). The hard part is propagating from $dhpreact$ back through the normalization operation to get $dhprebn$.

Setting up the variables for the derivation

Following the BatchNorm paper's notation: let x_i be the i -th element of $hprebn$, μ the mean over the batch, σ^2 the variance over the batch, $\hat{x}_i$ the normalized value, and y_i the final output. The compute graph has the i -th input feeding into the mean (one of n arrows), the variance (one of n arrows), and through its own normalization (its own arrow). When we apply the chain rule for dL/dx_i , all three of these paths contribute, and the sum is what we have to compute.

24. The full BatchNorm backward derivation

Chapter overview

This chapter walks through the analytical derivation step by step. The procedure is: compute the gradients with respect to the normalized values (easy), the variance (medium), and the mean (easy with a beautiful cancellation), then combine all three paths to get the gradient with respect to the input. We follow Karpathy's pen-and-paper derivation.

Step 1: gradient with respect to $\hat{x}$ (the normalized values)

Each y_i is just $\gamma * \hat{x}_i + \beta$. So $dL / d \hat{x}_i = \gamma * dL / d y_i$. We get this gradient on every element directly.

Step 2: gradient with respect to σ^2 (the variance)

Sigma-squared affects every $\hat{x}_i$ through the division $(\hat{x}_i - \mu) / \sqrt{\sigma^2 + \epsilon}$. There are n different $\hat{x}$ -hats, each contributing a gradient back to the same scalar sigma-squared. Summing over all of them and simplifying:

$$dL/d(\sigma^2) = \sum_i (dL/d \hat{x}_i) * (\hat{x}_i - \mu) * (-0.5) * (\sigma^2 + \epsilon)^{-3/2}$$

Step 3: gradient with respect to mu (the mean) - the beautiful cancellation

Mu also affects every $\hat{x}$ -hat (n contributions), and additionally affects sigma-squared (one more contribution). The full expression has two terms. But here a magical thing happens: when you plug in the special fact that mu is the average of the $\hat{x}_i$, the second term cancels exactly to zero. This dramatic simplification leaves us with just:

$$dL/d \mu = -(\sigma^2 + \epsilon)^{-1/2} * \sum_i dL/d \hat{x}_i$$

Why does the cancellation happen

The reason is structural. Mu was defined as the mean of the $\hat{x}_i$'s, which means it sits in a specific algebraic relationship to them. When we compute $d(\sigma^2)/d \mu$ using the chain rule while respecting that mu is itself a function of the $\hat{x}$'s, the contribution from mu's effect on sigma-squared vanishes. The result is one of those moments in derivations where the algebra *tells you* something is geometrically right — the variance, by construction, is invariant to small shifts in the mean.

Step 4: combining the three paths into dL/dx_i

Each input $\hat{x}_i$ has three paths to the loss: through its own normalization, through the mean, and through the variance. Sum them all:

$$\begin{aligned} dL/dx_i &= (dL/d \hat{x}_i) * (\sigma^2 + \epsilon)^{-1/2} && \# \text{ direct} \\ &+ (dL/d \mu) * (1/n) && \# \text{ through mean} \\ &+ (dL/d \sigma^2) * (2/n) * (\hat{x}_i - \mu) && \# \text{ through variance} \end{aligned}$$

Substituting the expressions we derived in Steps 1-3, factoring out common terms, and rewriting in terms of $\hat{x}$ -hat (which we already have) produces the final formula in Chapter 25.

25. The compact one-line formula

Chapter overview

After all the substitution and factoring, the entire BatchNorm backward through the input reduces to a single line of code. This chapter presents the formula and explains how to read it.

Manual backward - dhprebn in one shot

```
dhprebn = bngain * bnvar_inv / n * (
    n * dhpreact
    - dhpreact.sum(0)
    - n / (n - 1) * bnraw * (dhpreact * bnraw).sum(0)
)
```

```
cmp('hprebn', dhprebn, hprebn)
```

Reading the three terms

- $n * \text{dhpreact}$ — the direct contribution, multiplied by n because the prefactor is $1/n$.
- $-\text{dhpreact}.\text{sum}(0)$ — the contribution from the mean. We subtract the sum of upstream gradients across the batch, because the mean's effect is the negative average.
- $-n/(n-1) * \text{bnraw} * (\text{dhpreact} * \text{bnraw}).\text{sum}(0)$ — the contribution from the variance. The $n/(n-1)$ factor comes from Bessel's correction, and the multiplication by bnraw reflects the fact that the variance's effect on each x_i depends on how far that x_i is from the mean.

The whole expression is wrapped in $\text{bngain} * \text{bnvar_inv} / n$, which comes out as a common factor of all three terms. The `cmp` verification reports approximate equality with a `maxdiff` around $9\text{e-}10$ — floating-point noise. The manual one-shot matches PyTorch's autograd-routed answer to within numerical precision.

What this saves us in practice

Eight intermediate gradient computations (Chapters 16-18) collapse to a single tensor operation. The forward pass also benefits because we no longer need to keep all those intermediate tensors around — we only need `bngain`, `bnvar_inv`, `bnraw`, and `dhpreact`, plus the constant `n`. Memory and compute both win. This is exactly how production frameworks implement BatchNorm's backward pass internally; we just hand-derived what PyTorch does behind the scenes.

Why this derivation is hard

It is worth pausing to acknowledge: this is the hardest single piece of math in the entire *makemore* series. Three things make it hard. First, the operation couples examples in the batch, so you cannot treat one row at a time. Second, the mean and variance create algebraic dependencies that require careful bookkeeping. Third, the final formula has to be vectorized across the 64 hidden neurons simultaneously, which means every sum and broadcast has to be in exactly the right direction. Getting this right on the first try is rare; Karpathy mentions that even he uses a working version as a reference.

26. Exercise 4: training without `loss.backward()`

Chapter overview

We have all the pieces. Exercise 4 puts them together into a complete training loop with no autograd dependency: the forward pass computes the loss, our manual backward pass computes all seven gradients, and the optimizer applies them. The entire loop runs inside `torch.no_grad()` for efficiency, because we are no longer asking PyTorch to track operations for autograd. This chapter walks through the full training script.

Cell - The complete training loop with manual backprop

```

max_steps = 200000
batch_size = 32
n = batch_size
lossi = []

with torch.no_grad():
    for i in range(max_steps):
        # minibatch
        ix = torch.randint(0, Xtr.shape[0], (batch_size,), generator=g)
        Xb, Yb = Xtr[ix], Ytr[ix]

        # forward pass
        emb = C[Xb]
        embcat = emb.view(emb.shape[0], -1)
        hprebn = embcat @ W1 + b1
        # BatchNorm (collapsed form)
        bnmean = hprebn.mean(0, keepdim=True)
        bnvar = hprebn.var(0, keepdim=True, unbiased=True)
        bnvar_inv = (bnvar + 1e-5) ** -0.5
        bnraw = (hprebn - bnmean) * bnvar_inv
        hpreact = bngain * bnraw + bnbias
        h = torch.tanh(hpreact)
        logits = h @ W2 + b2
        loss = F.cross_entropy(logits, Yb)

        # manual backward pass - 20-ish lines
        # ... (Exercise 1 derivations, reorganized for efficiency) ...

        # update
        lr = 0.1 if i < 100000 else 0.01
        for p, grad in zip(parameters, grads):
            p.data += -lr * grad

```

What is and is not in the loop

- **No `loss.backward()` call.** The line that started this whole journey is gone. The gradient computation happens explicitly in our hand-written backward block.
- **No `p.grad = None` step.** We are not using PyTorch's gradient accumulators at all. Each gradient is computed fresh and stored in a local variable.
- **No `requires_grad=True` needed.** Since we are not using autograd, we do not need to mark parameters for tracking. The `torch.no_grad()` wrapper makes this explicit.
- **The update uses `grad` directly** instead of `p.grad`. We zip parameters with our manually computed gradients.

Why `torch.no_grad()` at the outer level

Without `torch.no_grad()`, PyTorch would still build the computation graph for every operation in the forward pass (just in case we wanted to call backward later). Building that graph costs memory and time, both of which are wasted if we are computing gradients manually. `torch.no_grad()` turns the graph-building off, making the whole forward pass faster and lighter. This is also what production inference paths use, since they never need gradients.

27. Verification and the final loss

Chapter overview

Karpathy runs the training loop for 200,000 steps with the same learning rate schedule and batch size as Part 3, calibrates the BatchNorm running statistics at the end, and evaluates on the train and validation splits. This chapter records the result and verifies that manual backprop produces the same final loss as autograd.

The result

Method	Train loss	Val loss
Part 3 (with <code>loss.backward()</code>)	~ 2.07	~ 2.10
Part 4 (manual backprop)	2.0719	2.1162

Essentially identical to Part 3. This should be unsurprising: we are doing the same computation, just with the backward written by hand. The exercise was never about improving the loss; it was about understanding what the loss-improving machinery actually does. Sampling from the trained model also produces the same quality of name-like gibberish we saw in Part 3.

What it feels like to delete `loss.backward()`

Karpathy explicitly notes the moment in the lecture: “It feels amazing to say that.” After spending the entire video deriving every gradient by hand, deleting the line `loss.backward()` from the training loop and seeing the network train to the same loss is the emotional payoff for the entire exercise. The network is now training purely on math we derived ourselves.

28. Diagnostic cards: common manual-backprop bugs

Chapter overview

When your manual gradient does not match PyTorch's, the bug is almost always one of a small set of common errors. This chapter consolidates them into stand-alone diagnostic cards. Keep these handy — you will encounter every one of them at least once when you try this exercise on your own networks.

Bug 1 · Missing sum along broadcast axis

Symptom	Manual gradient has the wrong shape (an extra dimension), or has correct shape but wrong values that look like one row of the right answer.
Diagnosis	When a tensor is broadcast in the forward pass, its backward needs a sum over the broadcast axis. Forgetting the sum leaves the gradient with one row per batch element instead of one row total.
Action	Identify every broadcast in the forward pass and add <code>.sum(axis, keepdim=True)</code> to the corresponding backward computation. The shape constraint will tell you which axis.

Bug 2 · Missing keepdim

Symptom	Gradient shapes look almost right but broadcasting in a downstream computation does something unexpected. <code>cmp</code> reports a huge maxdiff in a place that should be tiny.
Diagnosis	A sum or mean without <code>keepdim=True</code> collapses a dimension. The result has shape <code>(n,)</code> instead of <code>(n, 1)</code> , and subsequent broadcasts go in the wrong direction.
Action	Add <code>keepdim=True</code> to every reduction that maintains a broadcast-friendly shape. The rule of thumb: if the input had a singleton dimension, the gradient must too.

Bug 3 · Missing += for multi-path tensors

Symptom	Manual gradient matches PyTorch on its first contribution but is missing magnitude or sign of a second contribution.
Diagnosis	When the same tensor is used in multiple operations in the forward pass, every operation contributes a separate gradient that must be summed. Overwriting with <code>=</code> instead of accumulating with <code>+=</code> loses all but the last.
Action	Trace every forward tensor and count how many downstream operations use it. The backward must accumulate that many contributions with <code>+=</code> .

Bug 4 · Wrong transpose in matmul backward

Symptom	Shape error when computing the gradient, or shapes match by coincidence but values are wrong.
Diagnosis	Forgot to transpose one of the operands in $dA = dD @ B.T$ or $dB = A.T @ dD$. Easy to do because both forms involve a transpose and the directions look similar.
Action	Use the shape-matching shortcut from Chapter 14. The target shape (the shape of dA or dB) uniquely determines which operand is transposed.

Bug 5 · Forgot scatter-add for repeated indices

Symptom	Gradient on the embedding matrix C has zeros in rows that should have nonzero values, or has only the gradient from the last occurrence.
Diagnosis	The embedding lookup uses the same row of C for multiple inputs (the letter 'a' is used many times). Overwriting $dC[ix] = demb[k, j]$ instead of $+=$ loses all but the final contribution.
Action	Always use $+=$ when scattering gradients back through an indexing operation. Or use PyTorch's <code>index_add_</code> for the vectorized form.

Bug 6 · Bessel's correction inconsistency

Symptom	BatchNorm backward gradient is off by a factor close to 1 (typically $n/(n-1)$ or its inverse).
Diagnosis	PyTorch's <code>var</code> defaults to <code>unbiased=True</code> (divides by $n-1$), but writing <code>(diff ** 2).mean()</code> or hand-implementing variance with <code>/n</code> uses the biased estimator instead.
Action	Pick one convention and use it everywhere. The notebook uses $1/(n-1)$ in both forward and backward; PyTorch's default matches that. If you switch, switch consistently in both directions.

29. Summary and what Part 5 will introduce

Chapter overview

Part 4 is the most procedurally dense lecture in the series. This chapter consolidates the patterns you should have internalized and previews what the next lecture introduces.

The seven things to take away

- **1. Backprop is a leaky abstraction.** Autograd works, but it can produce technically-correct-but-useless gradients (vanishing, saturated, dead). Understanding manual backprop is the defense.
- **2. Six operation patterns cover almost everything.** Element-wise (apply scalar derivative), reduction (sum becomes broadcast in backward), broadcast (becomes sum in backward), matrix multiplication (transpose pattern), indexing (becomes scatter-add), and explicit operations like exp and tanh (reuse forward output).
- **3. The shape-matching shortcut for matmul backward.** Know the target shape, know the available tensors, find the unique combination of transposes that produces the target shape. Saves you from re-deriving matrix calculus.
- **4. Multi-path tensors require summed gradients.** Any tensor used multiple times in the forward must accumulate gradients from each use in the backward. Always +=, never =.
- **5. Cross-entropy backward is (softmax - one_hot) / N.** Three lines replace seven derivations. Read it as a push-and-pull system: pull the correct class up, push the incorrect ones down, magnitude proportional to error.
- **6. BatchNorm backward is one line.** The analytical derivation is hard but the result is compact. In production, this is how every framework implements BatchNorm's backward.
- **7. The cmp discipline.** Always verify manual gradients against autograd with exact, approximate, and maxdiff checks. Tiny floating-point differences are fine; large ones reveal bugs.

Where Part 5 picks up

Part 5 of the makemore series (*Building makemore Part 5: Building a WaveNet*) finally moves beyond the MLP. The architecture introduced is WaveNet, an autoregressive model originally designed by DeepMind for audio generation but adaptable to character-level language modelling. It uses dilated causal convolutions to process longer contexts than our 3-character window, stacks deeper than our 6-layer MLP, and produces noticeably better names. After three lectures of polishing the same MLP, we finally graduate to a meaningfully different architecture — with all the diagnostic and manual-backprop intuition built up across Parts 3 and 4 to support us.

The single thing to remember if you remember nothing else

Backpropagation is not a mystery. It is the chain rule applied tensor-by-tensor, with broadcasting becoming summation and indexing becoming scatter-add. Every modern neural network you encounter, no matter how complex, is built out of operations whose backward you can derive in five minutes using the patterns from this lecture. The notebook trained without `loss.backward()` by the end. So can you, on any network you build from this point forward.

30. Glossary

This glossary defines every technical term used in the handbook in a single, scannable place. Look up anything you do not recognize here first.

Autograd. Short for “automatic differentiation.” PyTorch's system for tracking operations on tensors and computing gradients automatically via `backward()`. Part 4's exercise is to replace it with manually derived gradients.

Backprop / backpropagation. Computing gradients of the loss with respect to every parameter (and intermediate tensor) of a neural network by applying the chain rule from the output back to the inputs.

Bessel's correction. Using $n - 1$ instead of n in the denominator of a sample variance. Provides an unbiased estimator of the population variance. PyTorch's `torch.var` defaults to this convention.

Broadcast in forward = sum in backward. The fundamental rule: when a tensor is replicated along some axis to make a forward operation work, its gradient in the backward pass is the sum over that same axis.

Chain rule. The calculus identity that says the derivative of a composition $f(g(x))$ is $f'(g(x)) * g'(x)$. Generalizes to tensor operations via matrix and element-wise products.

cmp utility. The small comparison function defined in the notebook that checks a manually-computed gradient against PyTorch's autograd result for exact equality, approximate equality, and maximum element-wise difference.

Cross-entropy backward (analytical form). `dlogits = (softmax(logits) - one_hot(Y)) / N`. The compact analytical gradient of the cross-entropy loss with respect to the logits, replacing the chain of seven derivations through softmax, log, and NLL.

Element-wise backward. When the forward operation is element-wise (tanh, log, exp, square), the backward is the scalar derivative of that operation applied element-wise to the forward input, multiplied by the upstream gradient.

Forward output reuse. An optimization where the backward pass uses the already-computed forward output instead of recomputing it. Applies to exp (output equals derivative), tanh (derivative is $1 - \text{output}^2$), and softmax.

Gather / scatter. *Gather* selects entries from a tensor based on indices (what indexing does in forward). *Scatter* writes entries into a tensor at specified positions (what the backward of indexing does). *Scatter-add* accumulates contributions at each position rather than overwriting.

Indexing backward = scatter-add. When the forward uses integer indexing to select rows/columns, the backward must scatter the upstream gradient back to those positions. Repeated indices require accumulation, not overwriting.

Leaf tensor. A PyTorch tensor created directly (not produced by an operation on other tensors). Parameters and inputs are leaves. By default, only leaf tensors keep their gradients after `backward()`.

Leaky abstraction. Karpathy's term for autograd. Pretends to hide the backward pass but in fact exposes the user to bugs (saturated activations, dead neurons, vanishing gradients) that are only debuggable with knowledge of what is hidden.

Logit. The raw, pre-softmax output of a classifier. Real-valued, can be positive or negative.

Manual backprop. Deriving and implementing the backward pass of a neural network by hand, without relying on autograd. The exercise of Part 4.

Matrix multiplication backward. For $D = A @ B + C$, the gradients are $dA = dD @ B.T$, $dB = A.T @ dD$, $dC = dD.sum(0)$. Derivable by writing out a small example.

Maxdiff. The maximum absolute element-wise difference between two tensors. The third value printed by `cmp`. Tiny maxdiff (1e-9 or below) is floating-point noise; large maxdiff is a bug.

Multi-path tensor. A tensor used as input to more than one operation in the forward pass. Its gradient in the backward is the sum of contributions from every use.

Numerical stability (logit max subtraction). Subtracting the per-row maximum from the logits before exponentiating. Prevents overflow in `exp`. The gradient on the max branch should be essentially zero, and indeed comes out very small.

One-hot encoding. A vector representation where one entry is 1 and all others are 0. The gradient of cross-entropy involves subtracting a one-hot for the correct class from the softmax probabilities.

Push-and-pull intuition. Reading the cross-entropy gradient as a system of forces: pull the correct class logit up, push every incorrect class down, with magnitude proportional to the prediction error. The forces sum to zero per example.

`retain_grad()`. PyTorch method that opts a non-leaf tensor in to having its gradient stored after `backward()`. Required for verifying manual gradients on intermediate tensors.

Scatter-add. Writing values into specified positions of a tensor while accumulating (`+=`) at repeated positions. The backward of indexing requires this when the same index appears multiple times.

Shape-matching shortcut. The practical trick for deriving matrix-multiplication backward: figure out what shape the result must be, identify the available tensors, and find the unique combination of transposes that produces that shape.

Softmax. $\text{softmax}(x)_i = \exp(x_i) / \sum_j \exp(x_j)$. Turns logits into a probability distribution. Its backward in cross-entropy simplifies dramatically when fused with the log-likelihood loss.

Sum in forward = broadcast in backward. The dual of the broadcast rule: when a forward operation sums across an axis, the backward replicates (broadcasts) the upstream gradient back across that axis.

torch.no_grad(). Context manager that disables autograd graph construction inside its block. Used in Exercise 4 because manual backprop does not need PyTorch's graph; turning it off saves memory and compute.

Transpose pattern in matmul backward. The observation that matrix-multiplication backward produces two matrix multiplications, each involving one of the original operands transposed. Memorizable but more easily reconstructed by shape matching.

Variance backward. The trickiest piece of BatchNorm backward. The local derivative of the variance with respect to each input is $2 * (x_i - \mu) / (n - 1)$, but the full path back to dx_i requires combining the variance contribution with the mean and direct contributions.

View backward = view. The backward of `.view()` is just another `.view()` back to the original shape. `.view()` never copies data, so its backward is trivially free.

End of Part 4. Part 5 finally moves beyond the MLP into the world of dilated causal convolutions and the WaveNet architecture — deeper, with longer effective context, and noticeably better samples. Three lectures of polishing the same network ends here. The diagnostic toolkit from Part 3 and the gradient-level intuition from Part 4 come with us into the new architecture.